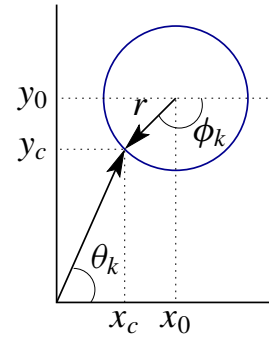




Illustrating the SIS steps with an example

- The previous lesson introduced an example that we will use to help illustrate the SIS particle-filter method.
- We consider is a car driving around a circular track having radius $r = 200$ m.
- Velocity is nearly constant at $\omega = 2^\circ$ per sample, but may be perturbed by zero-mean additive Gaussian noise having $\sigma_w = 10^\circ$ that models bumps, maneuvers, etc.
- The sensor is located at $(0, 0)$ and the center of the track is located at $(x_0, y_0) = (500, 500)$.
- The sensor measures the angle to the car, θ_k (i.e., $z_k = \theta_k$) with zero-mean additive Gaussian noise having $\sigma_v = 5^\circ$.
- We wish to estimate ϕ_k (i.e., $x_k = \phi_k$).



Code to create an input/output dataset

- Before we do anything else, we “collect” a simulated dataset of input/output data from the system model.

```
% Define constants describing problem
meanX0 = 50*pi/180;          sigmaX0 = 10*pi/180;
sigmaW = 10*pi/180;          sigmaV = 5*pi/180;
x0 = 500; y0 = 500;          r = 200;
omega = 2*pi/180;            nIter = 101;

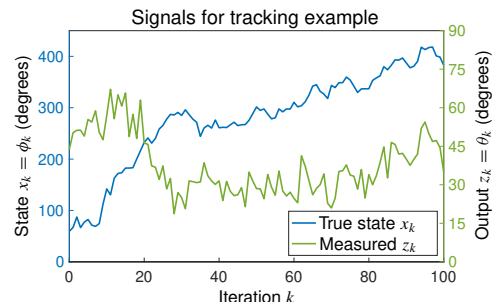
% Simulate the model to create a sequence of observations
x = zeros(nIter+1,1);        z = zeros(nIter,1);
wk = randn(nIter,1)*sigmaW;  vk = randn(nIter,1)*sigmaV;

x(1) = meanX0 + randn(1)*sigmaX0; % true initial state
for k = 1:nIter
    x(k+1) = x(k) + omega + wk(k);
    z(k) = atan((r*sin(x(k))+y0)./(r*cos(x(k))+x0)) + vk(k);
end
x = x(1:nIter); % Trim to iterations 0:nIter-1
```



Example of input/output dataset

- The car's initial position is randomly generated at $\phi_0 \sim \mathcal{N}(50^\circ, (10^\circ)^2)$.
 - In this case, at around 59° .
- The constant rotation, $\omega = 2^\circ$ per iteration, causes ϕ_k to tend to increase over time.
- The process noise w_k causes ϕ_k both to increase or decrease over time.
- Note that the state $x_k = \phi_k$ varies between 59° and about 420° .
- The output $z_k = \theta_k$ varies only between about 19° and 67° .
- Next, we implement SIS...





Sequential Importance Sampling implementation

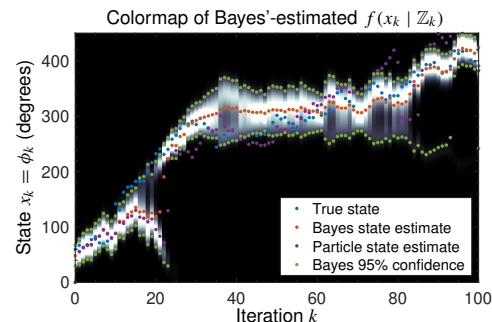
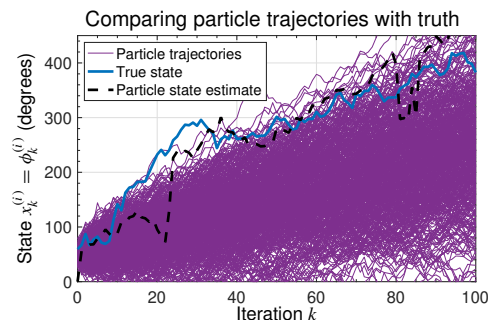
```
% Create storage for the particles
nPart = 500;          xHat = zeros(nIter,1);
xp = zeros(nPart,nIter);  wp = zeros(nPart,nIter);

% Run Sequential Importance Sampling (SIS) basic algorithm
xp(:,1) = meanX0 + sigmaX0*randn(nPart,1);  wp(:,1) = 1/nPart;
for k=2:nIter
    for cp=1:nPart
        % Draw new particle from q(x{k}|x{k-1},z{k}) = f(x{k}|x{k-1})
        xp(cp,k) = xp(cp,k-1) + omega + sigmaW*randn(1);
        % Compute the expected value of the measurement given x{k}
        zp = atan((r*sin(xp(cp,k))+yo)/(r*cos(xp(cp,k))+xo));
        % Compute f(z{k}|x{k})
        likelihood = normpdf(z(k),zp,sigmaV);
        % Compute unnormalized weight: wptilde(cp,k) = wp(cp,k-1) * f(z{k}|x{k})
        % ... * f(x{k}|x{k-1}) / q(x{k}|x{k-1},z{k}) where this last ratio = 1
        wp(cp,k) = wp(cp,k-1)*likelihood;
    end
    wp(:,k) = wp(:,k)/sum(wp(:,k)); % Normalize weights
    xHat(k) = sum(wp(:,k).*xp(:,k)); % Compute mean based on particles
end
```



Sample output from SIS method

- The particles, with true state trajectory overlaid, are shown.
- Particles represent many possible trajectories to choose from.
- Low particle density near true state; state estimate is similar to Bayes' optimal.
- Need to dig deeper to see if anything is going wrong with our estimator.



Summary

- In this lesson, you learned how to implement the system model for the example problem in Octave code to generate a dataset of observations.
- You also learned how to implement the SIS algorithm in Octave code.
- Running the code, we saw that many of the particle trajectories are quite different from the true state trajectory.
- The weighted mean of these trajectories doesn't seem too bad, compared with the Bayes' optimal solution computed by "brute force," but we still need to look more closely at the output of the SIS algorithm to make sure that it is working as intended.